SoK: Static Configuration Analysis in Infrastructure as Code Scripts

Pandu Ranga Reddy Konala
School of Computing and Mathematical Sciences
University of Waikato
Hamilton, New Zealand 3240
Email: pkonala@waikato.ac.nz

Vimal Kumar
School of Computing and Mathematical Sciences
University of Waikato
Hamilton, New Zealand 3240
Email: vkumar@waikato.ac.nz

David Bainbridge
School of Computing and Mathematical Sciences
University of Waikato
Hamilton, New Zealand 3240
Email: davidb@waikato.ac.nz

2023 IEEE International Conference on Cyber Security and Resilience (CSR) | 979-8-3503-1170-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/CSR57506.2023.10224925

***Abstract*—This SoK paper presents findings from a survey conducted on the current state of tools and techniques used in the static configuration analysis of Infrastructure as Code (IaC). Our findings highlight the increasing importance of ensuring the quality of IaC scripts through techniques such as detecting code and security smells. Our findings reveal that regular expressions are widely used, but this may not be a long-term or fully automated solution for detecting smells. Additionally, our study found that the majority of the tools and techniques are developed for infrastructure provisioning, rather than configuration management and image building. This raises concerns because configuring software is a high-risk task, with malicious actors constantly targeting software systems. Therefore, it is crucial for researchers to develop efficient and advanced techniques for detecting defects in configuration management and image building. The aim of this paper is to provide a detailed overview of the current state of research in this field, and to identify areas for future development.**

Index Terms—devops, devsecops, sok, survey, empirical study, infrastructure as code, code, security, smells, defects, static configuration analysis

I. INTRODUCTION

The growing adoption of IaC practices has brought many benefits to organisations, but it has also introduced new risks in the form of "code smells" and "security smells" in the scripts that may indicate potential vulnerabilities. The term code smells [1] can be described as a potential design problem in the code that could lead to difficulties in maintaining or extending the codebase. Code smells do not necessarily indicate a bug or an error in the code, but rather a potential problem that may need to be addressed. Similar to code smells, security smells [2] are characteristics of code that indicate a potential security vulnerability. They are indicators that there may be a deeper problem in the code that could lead to security risks or threats to the system. Security smells may be caused by incorrect or incomplete security implementations, or by a lack of awareness of security risks.

These smells arise from patterns or anomalies in the code that may indicate design problems, or security risks. If left unaddressed, these smells can lead to misconfigurations, which can cause unexpected behavior of the infrastructure and result in vulnerabilities. To combat this, researchers have developed various techniques and tools to detect these smells in IaC scripts.

In this paper, we present a detailed examination of the latest advancements in the field of static configuration analysis in IaC. We have carried out an extensive review of the literature to gather information about the different techniques and tools that have been proposed and built for this form of analysis, both academically and commercially. Additionally, we highlight the areas that require further investigation to enhance the effectiveness and efficiency of detecting configuration defects in IaCs. Lastly, we discuss the potential future trajectory of research in this field. To guide our work, we formulated the following key questions:

- **Q1** What are the existing research and tools available for static configuration analysis of IaC scripts?
- **Q2** What are the techniques used by existing tools to detect the smells?
- **Q3** What insights can be gained from examining the categories of IaC targeted by existing tools?
- **Q4** Which specific aspects or characteristics of IaC are addressed by existing tools?

II. BACKGROUND

Configuration management has evolved from the ad-hoc usage of shell scripts to the advanced state it is in today, driven by the advent of DevOps principles. One of the key components of DevOps is IaC [3]. IaC is a method of managing and provisioning IT infrastructure through the use of code, rather than manual configuration [3]. This is typically achieved through the use of a wide range of programs, scripts, or other code-based automation tools. To effectively manage the infrastructure, the concepts of IaC are broken down into three distinct categories, each with its own set of specialized software [3]. These categories are:

1 **Infrastructure Provisioning** deals with the automated provisioning and management of physical and virtual servers, storage, and networking devices. This includes the physical setup of servers, installing the Operating System, and configuring the network. Some examples of this type of software are *Terraform*, *TOSCA*, and *Cloudify*.

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 21,2026 at 21:19:51 UTC from IEEE Xplore. Restrictions apply.

2 **Configuration Management** deals with the automated configuration and management of software and services on servers. This includes installing software, configuring settings, and applying patches. Some examples of software in this category are *Ansible*, *Chef*, and *Puppet*.
3 **Image Building** deals with the creation of machine images that can be used to quickly instantiate new servers. This includes creating a base image of an operating system and capturing the image as a template that can be used to create new servers or instances. Examples of image building software include *HashiCorp Packer* and *Docker*.

Note that these categories can overlap and complement each other. These IaC tools can save time compared to manual configurations, but they also have their limitations. One of the most critical limitations of IaC software is the difficulty of detecting and analyzing code smells and security risks. As a result, IaC experts need to use additional resources to manually run bespoke checks they have developed or use specialized tools. If these code and security smells go unchecked, it heightens the risk of cyber attacks. To address the issues of defects in IaC, methods similar to those used in software development can be implemented in the form of static configuration analysis and dynamic configuration analysis. One of the most common and resource efficient methods is static configuration analysis by IaC scripts which is the focus of this paper.

Static configuration analysis involves evaluating configuration scripts to detect code and security smells without actually executing the scripts. This allows IaC experts to identify and address risks before deployment. The process can be performed manually or with the use of automated tools that scan the scripts for a specific smell category. This type of analysis focuses on the syntax of IaC. It is advantageous for organisations as it does not require deploying infrastructure, thus saving on costs and time for creating isolated test environments.

III. FINDINGS

Q1 What are the existing and tools available for static configuration analysis of IaC scripts?

A comprehensive review of existing work related to smells, in the field of IaC was undertaken. This review included peer-reviewed publications, whitepapers as well as tools that were widely used but not written about in a publication, and dataset analysis. We categorized the work into two groups:

Empirical studies: Four studies were identified that conducted empirical research on IaC, all of which analyzed data to investigate questions related to the topic.

Jiang et al. [4] conducted a research study that looked at changes made to IaC scripts. They discovered that these scripts are frequently modified by regular developers rather than experts in infrastructure, which can lead to poor code quality and a greater risk of security misconfigurations. The study highlighted the need for automated methods of detecting these risks and called for further research into tools and techniques for accomplishing this.

Guerriero et al. [5] conducted 44 interviews with IT professionals to gather information on their practices for developing IaC and the software they commonly use. The results of this research show that Ansible is the most widely used software among those surveyed.

Sharma et al. [6] conducted a preliminary analysis of the quality of configuration code by analyzing 4,621 *Puppet* repositories containing 142,662 *Puppet* files and over 8.9 million lines of code for configuration issues. They found that security smells which belong to a specific category often tend to coexist with security smells belonging to another smell category.

Chiari et al. [7] published an SoK paper in which they reviewed various static configuration analysis tools. However, their study only focused on tools that were backed by publications, and thus did not include techniques used in practice that may not have been formally published. This approach may lead to potential hazards as it can overlook widely-used and effective tools that lack formal publications or academic recognition. Therefore, it is important to consider a range of tools, including those that are not extensively detailed in published articles, to gain a comprehensive understanding of the available options and their capabilities.

Techniques / Tools: The topic of detecting and addressing code and security smells in IaC scripts has gained attention recently and is increasingly being studied. Consequently, a number of tools and techniques to detect these issues in IaC scripts have emerged. The published work either describes new approaches and enhancements to existing IaC tools or provides them with new capabilities. We identified 17 tools that are supported by publications and additionally, 7 tools that are widely used, although they are not extensively detailed in a published article. We include the results of our analysis of these tools in Table I. This table provides a comprehensive overview of the key characteristics of these tools, which can be interpreted to gain insights into their capabilities and limitations. It includes several useful parameters which are described below:

- **Tool Name** is the tool's name given by the authors.
- **Year** in the table refers to the initial release year of the tool by its authors.
- **Target Property** represents the overall objective that the tool aims to accomplish.
- **Detection Technique** refers to the method employed by the tool to identify and achieve the target property.
- **Availability** indicates whether the tool is publicly available as an open source, proprietary, or hybrid (a combination of open source and proprietary) software.
- **DevOps Integration** indicates whether or not the tool is capable of integration with the DevOps workflow of organizations.
- **Script Analysis Location** parameter is utilized to determine the location where IaC scripts are being processed in order to identify the target property. This parameter, which is mentioned in the documentation and source code of the tools, can take four distinct values: "Onsite", "Offsite", "Both", and "Unknown". The "Onsite" location

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 21,2026 at 21:19:51 UTC from IEEE Xplore. Restrictions apply.

refers to the local or organizational infrastructure where the analysis is conducted, while the "Offsite" location indicates that the analysis is done on third-party infrastructure. The "Both" location option allows the user to select either the "Onsite" or "Offsite" location. Finally, the "Unknown" location is assigned when the analysis location cannot be identified, usually due to the tool being proprietary or complex to test.

- **Supported DevOps Tools** indicates the software tools that the IaC software is designed to work with.

IV. SYSTEMIZATION OF RESULTS

Q2 What are the techniques used by existing tools to detect the smells?

To organize our research findings, we have systemized our review based on the security smell detection techniques used. Fig.1 provides an overview of the tools we have studied in relation to these techniques grouped by IaC category, which we now explain in detail.

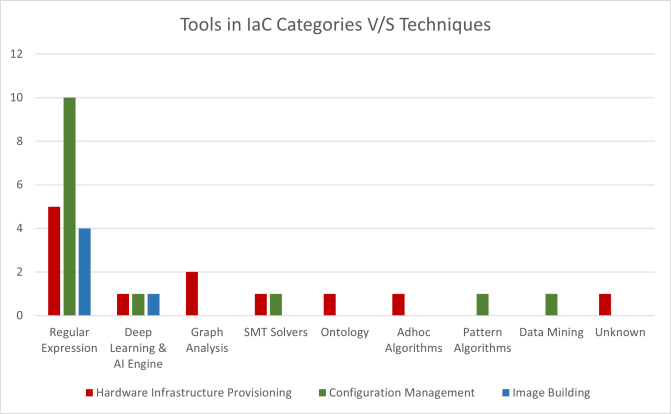

Fig. 1. Number of tools VS techniques

- **Regular Expressions:** From Fig.1 it can be seen that regular expressions are the most commonly used method for detecting security smells in IaC. This technique is popular due to its ease of implementation and the ability to customize the keywords quickly. It helps DevOps teams quickly identify security smells. Rahman et al. [2] used this technique to develop the tools SLAC and SLIC and further expanded their research to create the tool ACID. ACID examines *git* commit history, such as differences between commit messages, using a set of regular expression rules. And the goal is to identify patterns in version control history that may indicate the presence of security smells in the IaC scripts.
  Saavedra et al. [15] developed a tool called GLITCH, designed to detect security smells in the scripts of configuration management tools. The tool is versatile and can convert scripts written in Ansible, Chef, and Puppet into an intermediate form represented by an Abstract Syntax Tree (AST). Then, similar to SLAC and SLIC, regular expressions are employed to detect security smells. The authors of GLITCH tool later compared it to SLAC and SLIC and found that their tool performed better than SLAC and SLIC regarding average precision and average recall with an improvement of 8 and 13 percentage points.
  In addition to identifying security smells, the use of regular expressions can also detect issues in the code, known as code smells. Tools such as Foodcritic [14] and Puppeteer [6] are used to identify code smells in Chef and Puppet scripts, respectively. Some common examples of code smells in both of these tools include improper alignment of code, long statements, and misplaced attributes. The authors of Foodcritic and Puppeteer later developed a tool called Cookstyle [12], which specifically focuses on code correctness and code smells in Chef. Cookstyle is a linting tool that helps DevOps teams to write better Chef cookbooks by detecting and automatically correcting issues in the code's style, syntax, and logic.
  Dai et al. [21] created a tool called SecureCode to detect security risks caused by the inclusion of shell scripts like Bash and PowerShell in configuration management scripts. The tool integrates with third-party applications like Shellcheck [30] and PSS-scriptAnalyzer [31] in the CI/CD pipeline to identify code smells. They evaluated the tool by analyzing 1,492 Ansible scripts from 45 GitHub repositories, and found that SecureCode detected 3,535 issues, with 116 being false positives.
  Commercially, many tools utilize regular expressions in various forms, such as pattern rules, to detect code and security smells. One widely used tool in the industry is KICS [17], which is integrated into the GitLab [32] platform's CI/CD pipeline to check for security issues before the scripts are executed in the runtime environment. Another tool, Semgrep [22], also offers similar capabilities as KICS in identifying security smells across different IaC platforms.
  Checkov [10] and *tfsec* [29] are tools that are designed to detect security smells in cloud infrastructure provisioning scripts, such as *terraform*. Both tools scan the configuration scripts and package files, as well as container images, for Common Vulnerabilities and Exposures (CVEs) using a technique called Software Composition Analysis (SCA). However, there is currently no publicly available information on the number of organisations using these tools or the results of any benchmarking tests.
  SonarQube [27] is used to check for code and security smells in software used in Hardware Infrastructure Provisioning and Image Building. The tool uses regular expressions to identify these issues and provides plugins for IDEs to assist code quality for IaC script that are being developed. While SonarQube is useful, there is no publicly available information on how widely they are adopted by organizations or how effective they are in detecting smells.
- **Deep Learning and AI Engine:** Borovits et al. [13] created a tool called DeepIaC to detect anti-patterns in Ansible scripts using a Convolutional Neural Network

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 21,2026 at 21:19:51 UTC from IEEE Xplore. Restrictions apply.

TABLE I
TOOLS DEVELOPED TO DETECT SMELLS FOR IAC SOFTWARE

Tool Name	Year	Target Property	Detection Technique	Availability	DevOps Integration	Script Analysis Location	Supported DevOps Tools
ACID[8]	2020	Anti-Patterns	Regular Expression	Opensource	No	Onsite	Puppet
BARREL[9]	2018	Deployment Plan Correction	Graph Analysis	Opensource	No	Onsite & Offsite	TOSCA
ψCheckov[10]	2021	Security Smells	Regular Expression	Hybrid	Yes	Onsite	Infrastructure Provisioning
ψCloudSploit[11]	2020	Security Smells	Unknown	Proprietary	Yes	Offsite	Infrastructure Provisioning
CookStyle[12]	2016	Code Smells	Regular Expression	Opensource	No	Onsite	Chef
DeepIaC[13]	2020	Anti-Patterns	Deep Learning	Opensource	No	Unknown	Ansible
Foodcritic[14]	2011	Code Smells	Regular Expression	Opensource	No	Onsite	Chef
GLITCH[15]	2022	Security Smells	Regular Expression	Opensource	No	Onsite	Ansible
Häyhä[16]	2021	Security Smells	Graph Analysis	Opensource	No	Onsite	AWS CloudFormation
ψKICS[17]	2020	Security Smells	Regular Expression	Opensource	Yes	Onsite & Offsite	Multiple Tools
Puppeteer[6]	2016	Code Smells	Regular Expression	Opensource	No	Onsite	Puppet
RADON[18]	2021	Anti-Patterns	Data Mining	Opensource	No	Onsite	Ansible
Rehearsal[19]	2016	Code Smells	SMT Solvers	Opensource	No	Onsite & Offsite	Puppet
SecGuru[20]	2014	Network Policies	SMT Solvers	Proprietary	No	Unknown	Microsoft Azure
SecureCode[21]	2020	Code Smells	Regular Expression	Proprietary	No	Unknown	Ansible
ψSemgrep[22]	2020	Security Smells	Regular Expression	Opensource	Yes	Offsite	Multiple Tools
SLAC[23]	2021	Security Smells	Regular Expression	Opensource	No	Onsite	Ansible, Chef
SLIC[2]	2019	Security Smells	Regular Expression	Opensource	No	Onsite	Puppet
ψSNYK IaC[24]	2023	Security Smells	AI Engine	Proprietary	Yes	Offsite	Multiple Tools
SODALITE[25]	2020	Multiple Properties	Ontology	Opensource	Yes	Onsite	TOSCA
Sommelier[26]	2017	Deployment Plan Correction	Adhoc Algorithms	Opensource	No	Onsite	TOSCA
ψSonarQube[27]	2016	Code Smells, Security Smells	Regular Expression	Hybrid	Yes	Offsite	Multiple Tools
TAMA[28]	2022	Pattern Instances	Pattern Algorithms	Opensource	No	Onsite	Ansible
ψtfsec[29]	2019	Security Smells	Regular Expression	Opensource	Yes	Onsite	Infrastructure Provisioning

ψ = Tools supported by IT Industry.

(CNN). They used the dataset collected by Rahman et al. [2] to train their model, but since the dataset was minimal, they also used an algorithm to artificially introduce bugs in the Ansible scripts. The tool is able to detect these artificial bugs with an accuracy in the range of 79–92%.

In industry, SNYK has incorporated the use of Artificial Intelligence into one of their products called SNYK IaC [24]. This tool aims to aid developers in identifying code smells in IaC and providing solutions to fix them. However, as the tool is proprietary and there is limited documentation available, it was not possible to conduct a thorough analysis of the AI algorithms and training data used by the tool.

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 21,2026 at 21:19:51 UTC from IEEE Xplore. Restrictions apply.

- **Graph Analysis:** Brogi et al. [9] developed a tool called BARREL to check the accuracy of TOSCA management plans. They used graph analysis on *TOSCA* node templates and found that representing them as a graph helps the DevOps team have a better understanding of the infrastructure. The tool verifies the plan's validity by creating a state representation of the entire infrastructure. The plan is considered valid if the state representation of the *TOSCA* templates follows the predefined labeled path.
  Lepiller et al. [16] developed a tool called Häyhä, which aims to detect a previously undiscovered type of vulnerability specific to IaC known as intra-update sniping vulnerabilities. These vulnerabilities occur from the absence of ordering in upgrading resources. To detect these vulnerabilities, Häyhä converts CloudFormation templates or scripts into dataflow graphs and showcases the vulnerabilities in a GUI.
- **SMT Solver:** Shambaugh et al. [19] developed a tool called *Rehearsal*. The tool utilizes the Z3 SMT solver to identify code smells in Puppet configuration scripts that violate the principles of determinism and idempotency by converting them into a formal language and translating them into SMT specifications. The tool was tested on a small dataset of 13 Puppet scripts and was able to detect code smells in 6 of them.
  The *SecGuru* tool aims to detect and fix faulty network access control policies in a Microsoft Azure cloud environment. Developed by Jayaraman et al. [20], it uses the Z3 SMT solver to encode the firewall's actions and policies into an SMT problem, and then solves it to determine whether to allow or block network packets. It has been tested with runtime network policies and is now integrated into the Azure cloud. The authors reported that their tool had a positive impact on managing network policies at scale.
- **Ontology:** Kumara et al. [25] developed a tool called SODALITE that focuses on multiple properties of TOSCA configuration scripts. This tool has various components for detecting code smells, security smells and a dedicated IDE to deployment validation, intended for staff with a "low code" skill base. SODALITE uses ontologies to detect code smells in TOSCA scripts, SPARQL, and RDF triple store to provide context to the detected code smells, and regular expressions to detect security smells. It also has an interactive IDE that uses Abstract Syntax Trees (AST) to graphically design *TOSCA* scripts using a drag-and-drop method. However, this tool is still under development and benchmark results are not yet available.
- **Ad hoc Algorithms:** In addition to BARREL, Brogi et al. [26] created a tool called Sommelier to verify the connections between entities in *TOSCA* scripts. The tool is designed to prevent undefined references that may occur due to poor coding practices, which can lead to errors during deployment. The authors have developed specific validation checks to prevent these undefined references from occurring.
- **Pattern Algorithms:** Rahman et al. [28] developed a tool called TAMA that uses pattern algorithms to identify and classify bugs in Ansible scripts. The tool uses techniques such as Assertion Roulette, Local Only Testing and Remote Mystery Guest, to classify bugs into seven categories: configuration, dependency, idempotency, logging, performance, security, and style. The tool was applied to a dataset of 4,831 Ansible scripts, and it found that 1.8 percent of the scripts contained patterns that matched one of the seven bug categories. TAMA also identified three unique testing patterns that are associated with the appearance of bugs, one of which is specific to Ansible test scripts.
- **Data Mining:** Dalla Palma et al. [18] created a defect prediction framework called RADON which detects anti-patterns in Ansible scripts. The tool processes 108 features related to IaC code and general software metrics to train a wide range of algorithms, such as feature selection, data balancing, data normalisation, and classification. They find that Isolation Forest is the best algorithm with a precision of 0.86 and recall of 0.77.
- **Unknown:** CloudSploit [11] is a popular tool among IT professionals for detecting security smells in provisioning tools. However, as the tool is proprietary, it is not possible to determine the specific techniques used for detecting security smells and its performance.

V. RESEARCH GAPS AND FUTURE RESEARCH

Q3 What insights can be gained from examining the categories of IaC targeted by existing tools?

We can see from Table I, that most research in this area has focused on infrastructure provisioning, with relatively less focus on configuration management and image building. Given the high risk associated with software configuration and the constant threat of malicious actors, it is important for researchers to develop new and efficient methods for identifying defects. Currently, there are only a limited number of tools available for automated detection of security smells in configuration management, such as SLIC, SLAC, GLITCH, Semgrep and KICS, which tend to rely on regular expressions to secure software configuration scripts.

From the past history of the development of software tools, we can say that when a new technology arrives, one is likely to find more open-source and academic products related to it than polished commercial tools. A sign of the maturity of a technology however, is the availability of abundant commercial and proprietary tools and services.

Based on the results presented in Table I, The IaC tools were classified into five categories based on DevOps integration and availability which depict the maturity of tool development as shown in Table II. These categories are C1, C2, C3, C4, and C5, with C1 representing the initial stage of IaC tool development, and C5 being the most advanced and mature stage.

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 21,2026 at 21:19:51 UTC from IEEE Xplore. Restrictions apply.

TABLE II
CATEGORIZATION OF TOOLS

Category	DevOps Integration	Availability
C1	No	Opensource
C2	No	Proprietary
C3	Yes	Hybrid
C4	Yes	Opensource
C5	Yes	Proprietary

The categories have distinct characteristics that signify the maturity of tool development. The majority of the tools (15) are in the C1 category. This is because we are looking at an active area of research where researchers have developed and made available their research outcomes as open-source tools, without any DevOps integration facility. There is some availability of tools in C2 and C3 categories while there are 4 tools in C4 category and 2 in the topmost C5 category. As this field of research matures we can expect to see more tools in the C4 and C5 categories.

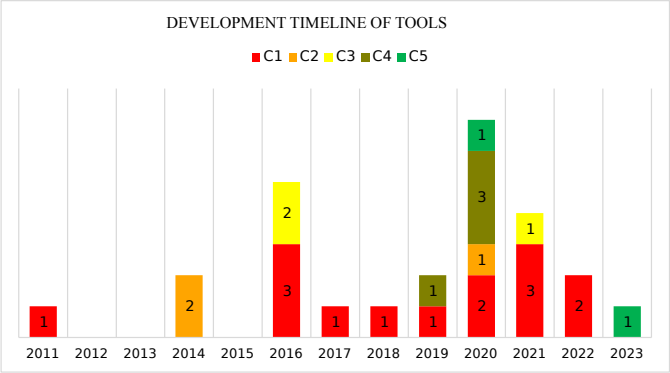

Fig. 2. The development timeline of the tools

To visualize the growth of IaC tools over time, we created a timeline as shown in the Fig.2 using data from table I and II. The timeline shows that the number of IaC tools released each year for various categories which has steadily increased since 2011, with a particularly large increase in 2020.

The increase in the number of IaC tools released in 2020 can be attributed to the COVID-19 pandemic, which resulted in a significant increase in demand for online services[33]. As more people began working remotely, the need for reliable, scalable, and secure cloud infrastructure grew, driving the development of new IaC tools to meet this demand. This growth is a testament to the importance of IaC tools in modern infrastructure management, as organizations continue to rely on cloud infrastructure to support their operations.

To provide a more detailed view of the development of IaC tools, we also created a heatmap of the maturity levels of the tools based on their script analysis location as shown in Fig.3. The heatmap provides additional context to this claim by showing that the C1 category has a high proportion of open-source tools with an onsite process location. This suggests that the tools in this category may be less sophisticated and less

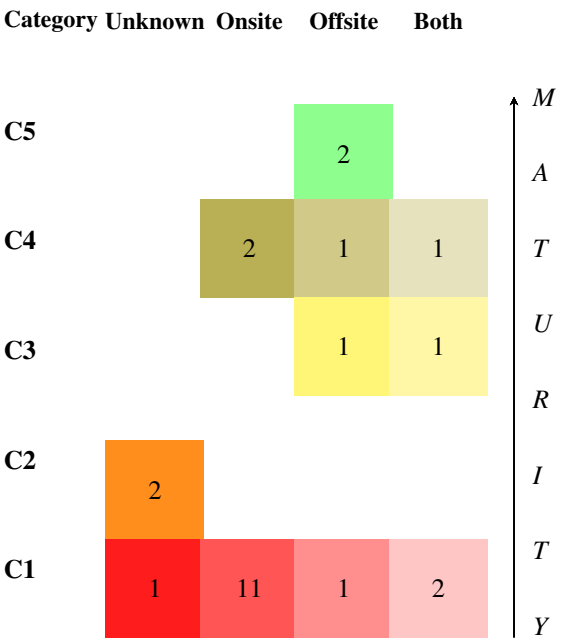

Fig. 3. Maturity levels of the IaC tools

mature than those in the other categories.

Over time, the development of IaC tools has gradually increased the number of tools released in the C2 category and beyond, as revealed by the timeline analysis. The accompanying heatmap provides additional context by highlighting a mix of open-source and proprietary tools in the category after C1. This suggests that the tools in this category may have advanced features and a more mature development status compared to those in the C1 category.

Together, the timeline and heatmap can be used to demonstrate the progression of tool development from the nascent C1 category to the more mature C5 category over time. This provides valuable insights into the evolution of IaC tool development and how it has progressed through different stages over time.

Q4 Which specific aspects or characteristics of IaC are addressed by existing tools?

The focus of current research on IaC is mainly on the identification of security smells through various techniques, with a large emphasis on the use of regular expressions. Finding security smells in IaC scripts is essential as these smells can lead to security misconfigurations. On the other hand, code smells, while indicating issues with code quality, do not pose the same level of risk. We believe that it is crucial to prioritize the identification and correction of security smells over code smells in IaC practices.

Regular expressions based techniques for detecting security smells in IaC scripts however have their limitations. Such techniques are unable to provide fully automated end-to-end solutions and can be circumvented in the case of attacks. Their efficiency is also affected by the inevitable changes in software and software development. Other techniques, such as deep

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 21,2026 at 21:19:51 UTC from IEEE Xplore. Restrictions apply.

learning, AI, data mining, and pattern matching algorithms, have also been proposed, but they require large datasets which can be challenging to obtain because of the sensitive nature of configuration scripts. In fact all published work in this area has relied on small, curated datasets from open-source software for their experimentation. This gap in data availability needs to be addressed to develop more comprehensive and robust data sets that can accommodate current and future vulnerability detection techniques.

VI. KEY TAKEAWAYS

In this section we highlight the key takeaways from our survey.

- IaC scripts are often modified by software developers rather than infrastructure experts, which can lead to poor code quality and a greater risk of security misconfiguration.[5]
- In-line with the findings of Guerriero et al. [5], we found Ansible to be the IaC component most commonly supported by IaC static analysis tools.
- The existing literature in this research area only focused on studying tools and techniques backed by publication while did not include techniques used in practice that may not have been formally published. This approach may lead to potential hazards as it can overlook widely-used and effective tools that lack formal publications or academic recognition.
- Regular expressions are currently the most widely used technique for detecting smells in IaC scripts.
- Our study revealed that there are only a handful of tools available for the automated detection of security smells in software configuration management, such as SLIC[2], SLAC[23], GLITCH[15], Semgrep[22] and KICS[17], which tend to rely on regular expressions. Further research in this area is especially warranted, particularly with regards to the types of techniques that can be utilized.
- Fig.2 and Fig.3 is indicative of the current research in this field still being in a nascent stage. There is a clear need for more sophisticated tools to detect smells in IaC scripts and address the risks associated with code smells and security smells. Overall, this study provides valuable insights into the current state of IaC tool development and highlights important areas for future research.

VII. CONCLUSION

This review illustrates the growing emphasis on ensuring the quality of IaC scripts through various static configuration analysis techniques, such as detecting code and security smells. While advancements have been made in this field, there is still room for improvement in terms of accuracy. Future research could involve developing mature automated tools to address risks and utilizing techniques specifically for the field of configuration management and image building that do not solely rely on regular expressions.

ACKNOWLEDGMENT

The authors would like to acknowledge funding from the New Zealand Ministry of Business, Innovation and Employment (MBIE) for project UOWX1911, Artificial Intelligence for Human-Centric Security.

REFERENCES

[1] P. A. Laplante, *What Every Engineer Should Know about Software Engineering (What Every Engineer Should Know)*. USA: CRC Press, Inc., 2007.

[2] A. Rahman, C. Parnin, and L. Williams, "The seven sins: Security smells in infrastructure as code scripts," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, 2019, pp. 164–175.

[3] R. Wang, *Infrastructure as Code, Patterns and Practices: With Examples in Python and Terraform*, ser. ITpro collection. Manning, 2022.

[4] Y. Jiang and B. Adams, "Co-evolution of infrastructure and source code: An empirical study," in *Proceedings of the 12th Working Conference on Mining Software Repositories*, ser. MSR '15. IEEE Press, 2015, p. 45–55.

[5] M. Guerriero, M. Garriga, D. A. Tamburri, and F. Palomba, "Adoption, support, and challenges of infrastructure-as-code: Insights from industry," in *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2019, pp. 580–589.

[6] T. Sharma, M. Fragkoulis, and D. Spinellis, "Does your configuration code smell?" in *2016 IEEE/ACM 13th Working Conference on Mining Software Repositories (MSR)*, 2016, pp. 189–200.

[7] M. Chiari, M. De Pascalis, and M. Pradella, "Static analysis of infrastructure as code: a survey," in *2022 IEEE 19th International Conference on Software Architecture Companion (ICSA-C)*, 2022, pp. 218–225.

[8] A. Rahman, E. Farhana, C. Parnin, and L. Williams, "Gang of eight: A defect taxonomy for infrastructure as code scripts," in *2020 IEEE/ACM 42nd International Conference on Software Engineering (ICSE)*, 2020, pp. 752–764.

[9] A. Brogi, A. Canciani, and J. Soldani, "Modelling and analysing cloud application management," in *Service Oriented and Cloud Computing*, S. Dustdar, F. Leymann, and M. Villari, Eds. Cham: Springer International Publishing, 2015, pp. 19–33.

[10] BridgeCrew, "Checkov." [Online]. Available: https://www.checkov.io/

[11] Aqua, "Cloudsploit." [Online]. Available: cloudsploit.com

[12] Chef, "Cookstyle." [Online]. Available: https://github.com/chef/cookstyle

[13] N. Borovits, I. Kumara, P. Krishnan, S. D. Palma, D. Di Nucci, F. Palomba, D. A. Tamburri, and W.-J. van den Heuvel, "Deepiac: Deep learning-based linguistic anti-pattern detection in iac," in *Proceedings of the 4th ACM SIGSOFT International Workshop on Machine-Learning Techniques for Software-Quality Evaluation*, ser. MaLTeSQuE 2020. New York, NY, USA: Association for Computing Machinery, 2020, p. 7–12. [Online]. Available: https://doi.org/10.1145/3416505.3423564

[14] J. Schwarz, A. Steffens, and H. Lichter, "Code smells in infrastructure as code," in *2018 11th International Conference on the Quality of Information and Communications Technology (QUATIC)*, 2018, pp. 220–228.

[15] N. Saavedra and J. F. Ferreira, "Glitch: Automated polyglot security smell detection in infrastructure as code," 2022. [Online]. Available: https://arxiv.org/abs/2205.14371

[16] J. Lepiller, R. Piskac, M. Schäf, and M. Santolucito, "Analyzing infrastructure as code to prevent intra-update sniping vulnerabilities," in *Tools and Algorithms for the Construction and Analysis of Systems*, J. F. Groote and K. G. Larsen, Eds. Cham: Springer International Publishing, 2021, pp. 105–123.

[17] Checkmarx, "Kics." [Online]. Available: https://www.kics.io/

[18] S. Dalla Palma, D. Di Nucci, F. Palomba, and D. A. Tamburri, "Within-project defect prediction of infrastructure-as-code using product and process metrics," *IEEE Transactions on Software Engineering*, vol. 48, no. 6, pp. 2086–2104, 2022.

[19] R. Shambaugh, A. Weiss, and A. Guha, "Rehearsal: A configuration verification tool for puppet," in *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 416–430. [Online]. Available: https://doi.org/10.1145/2908080.2908083

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 21,2026 at 21:19:51 UTC from IEEE Xplore. Restrictions apply.

[20] K. Jayaraman, N. Bjørner, G. Outhred, and C. Kaufman, "Automated analysis and debugging of network connectivity policies," *Microsoft Research*, pp. 1–11, 2014.
[21] T. Dai, A. Karve, G. Koper, and S. Zeng, "Automatically detecting risky scripts in infrastructure code," in *Proceedings of the 11th ACM Symposium on Cloud Computing*, ser. SoCC '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 358–371.
[22] Semgrep, "Semgrep." [Online]. Available: semgrep.dev
[23] A. Rahman, M. R. Rahman, C. Parnin, and L. Williams, "Security smells in ansible and chef scripts: A replication study," *ACM Trans. Softw. Eng. Methodol.*, vol. 30, no. 1, 1 2021. [Online]. Available: https://doi.org/10.1145/3408897
[24] SNYK, "Snykiac." [Online]. Available: https://snyk.io/product/infrastructure-as-code-security/
[25] I. Kumara, Z. Vasileiou, G. Meditskos, D. Tamburri, W.-J. Heuvel, A. Karakostas, S. Vrochidis, and I. Kompatsiaris, "Towards semantic detection of smells in cloud infrastructure code," 06 2020, pp. 63–67.
[26] A. Brogi, A. Di Tommaso, and J. Soldani, "Sommelier: A tool for validating tosca application topologies," in *Model-Driven Engineering and Software Development*, L. F. Pires, S. Hammoudi, and B. Selic, Eds. Cham: Springer International Publishing, 2018, pp. 1–22.
[27] Sonar, "Sonarqube." [Online]. Available: https://www.sonarsource.com/products/sonarqube/
[28] M. M. Hassan and A. Rahman, "As code testing: Characterizing test quality in open source ansible development," in *2022 IEEE Conference on Software Testing, Verification and Validation (ICST)*, 2022, pp. 208–219.
[29] aquasecurity, "tfsec." [Online]. Available: https://github.com/aquasecurity/tfsec
[30] koalaman, "Shellcheck." [Online]. Available: https://github.com/koalaman/shellcheck
[31] Microsoft, "Psscriptanalyzer." [Online]. Available: https://github.com/PowerShell/PSScriptAnalyzer
[32] GitLab, "About the one devops platform." [Online]. Available: https://about.gitlab.com/
[33] A. Feldmann, O. Gasser, F. Lichtblau, E. Pujol, I. Poese, C. Dietzel, D. Wagner, M. Wichtlhuber, J. Tapiador, N. Vallina-Rodriguez, O. Hohlfeld, and G. Smaragdakis, "The lockdown effect: Implications of the covid-19 pandemic on internet traffic," in *Proceedings of the ACM Internet Measurement Conference*, ser. IMC '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 1–18. [Online]. Available: https://doi.org/10.1145/3419394.3423658

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on January 21,2026 at 21:19:51 UTC from IEEE Xplore. Restrictions apply.